

Python Code

```

1  """
2  BuildDataset.py
3  =====
4  Builds an enriched, consolidated dataset from all bronze-raw sources.
5  Uses chunked processing to handle large files within memory constraints.
6  Outputs: enriched_dataset/dataset.parquet + MLTable
7  """
8
9  import subprocess, os, pandas as pd, numpy as np, pyarrow as pa
10 import pyarrow.parquet as pq
11 from datetime import datetime
12
13 ACCOUNT = "stukrealestateadk"
14 CONTAINER = "bronze-raw"
15 OUT_DIR = "enriched_dataset"
16 CHUNK_SIZE = 200_000
17
18 TRANSACTION_COLS = [
19     "Transaction_ID", "Price", "Transfer_Date", "Postcode",
20     "Property_Type", "New_Build", "Duration", "PAON", "SAON",
21     "Street", "Locality", "Town_City", "District", "County",
22     "PPD_Category", "Record_Status"
23 ]
24
25 os.makedirs(OUT_DIR, exist_ok=True)
26
27 # — 1. DOWNLOAD REFERENCE FILES
28
29 def download(blob_name, local_path):
30     if os.path.exists(local_path):
31         print(f" Already cached: {local_path}")
32         return
33     print(f" Downloading: {blob_name}")
34     subprocess.run([
35         "az", "storage", "blob", "download",
36         "--account-name", ACCOUNT,
37         "--container-name", CONTAINER,
38         "--name", blob_name,
39         "--file", local_path,
40         "--auth-mode", "login"
41     ], check=True)
42
43 print("=== Downloading reference files ===")
44 download("Bank Rate history and data Bank of England Database.csv",
45         "/tmp/bank_rate.csv")
46 download("consumerpriceinflationdetaileddreferencetables.xlsx",
47         "/tmp/cpi.xlsx")
48 download("series-050626 (1).csv",
49         "/tmp/gdp.csv")

```

```

47 download("LiveTable253.xlsx",
48 "/tmp/livetable.xlsx")
49 download("202304.csv",
50 "/tmp/transactions_extra.csv")
51
52 # — 2. BUILD REFERENCE LOOKUP TABLES
53
54 print("\n=== Building reference tables ===")
55
56 # Bank Rate – forward-fill to get rate active on any given month
57 bank = pd.read_csv("/tmp/bank_rate.csv")
58 bank.columns = ["Date_Changed", "Bank_Rate"]
59 bank["Date_Changed"] = pd.to_datetime(bank["Date_Changed"], format="%d %b %y",
60 dayfirst=True)
61 bank = bank.sort_values("Date_Changed")
62 date_range = pd.date_range("1995-01-01", "2026-12-01", freq="MS")
63 bank_monthly = pd.DataFrame({"month_start": date_range})
64 bank_monthly = pd.merge_asof(bank_monthly, bank, left_on="month_start",
65 right_on="Date_Changed")
66 bank_monthly["Transaction_Month"] =
67 bank_monthly["month_start"].dt.to_period("M").dt.to_timestamp()
68 bank_monthly = bank_monthly[["Transaction_Month", "Bank_Rate"]].copy()
69 bank_monthly["Transaction_Month"] =
70 bank_monthly["Transaction_Month"].dt.strftime("%Y%m").astype(int)
71
72 # GDP – quarterly, forward-fill to monthly
73 gdp = pd.read_csv("/tmp/gdp.csv", skiprows=7, header=None, names=["Quarter",
74 "GDP_QoQ_Growth"], on_bad_lines='skip')
75 gdp = gdp[gdp["Quarter"].str.match(r"^\d{4} Q\d$", na=False)].copy()
76 gdp["GDP_QoQ_Growth"] = pd.to_numeric(gdp["GDP_QoQ_Growth"], errors="coerce")
77 def quarter_to_date(q):
78     year, qtr = q.split(" Q")
79     month = (int(qtr) - 1) * 3 + 1
80     return pd.Timestamp(int(year), month, 1)
81 gdp["month_start"] = gdp["Quarter"].apply(quarter_to_date)
82 date_range2 = pd.DataFrame({"month_start": pd.date_range("1995-01-01", "2026-
83 12-01", freq="MS")})
84 gdp_monthly = pd.merge_asof(date_range2.sort_values("month_start"),
85 gdp.sort_values("month_start"), on="month_start")
86 gdp_monthly["Transaction_Month"] =
87 gdp_monthly["month_start"].dt.strftime("%Y%m").astype(int)
88 gdp_monthly = gdp_monthly[["Transaction_Month", "GDP_QoQ_Growth"]].copy()
89
90 # CPI
91 cpi_raw = None
92 for skip in range(3, 20):
93     try:
94         df = pd.read_excel("/tmp/cpi.xlsx", sheet_name="Table 1",
95 skiprows=skip, header=0)
96         mask = df.iloc[:, 0].astype(str).str.match(r"^\d{4}", na=False)
97         if mask.sum() > 10:
98             cpi_raw = df[mask].copy()
99             cpi_raw.columns = [str(c) for c in df.columns]

```

```

89         break
90     except:
91         continue
92
93 if cpi_raw is not None:
94     date_col = cpi_raw.columns[0]
95     cpi_raw["Transaction_Month"] =
pd.to_datetime(cpi_raw[date_col].astype(str),
errors="coerce").dt.strftime("%Y%m")
96     cpi_raw = cpi_raw.dropna(subset=["Transaction_Month"])
97     cpi_raw["Transaction_Month"] = cpi_raw["Transaction_Month"].astype(int)
98     num_cols = cpi_raw.select_dtypes(include="number").columns.tolist()
99     if len(num_cols) >= 2:
100         cpi_lookup = cpi_raw[["Transaction_Month", num_cols[0],
num_cols[1]]].copy()
101         cpi_lookup.columns = ["Transaction_Month", "CPIH_Index", "CPI_Index"]
102     else:
103         cpi_lookup = cpi_raw[["Transaction_Month", num_cols[0]]].copy()
104         cpi_lookup.columns = ["Transaction_Month", "CPIH_Index"]
105 else:
106     cpi_lookup = None
107
108 # LiveTable
109 xl = pd.ExcelFile("/tmp/livetable.xlsx")
110 fy_sheets = [s for s in xl.sheet_names if s.startswith("FY_")]
111 live_frames = []
112 for sheet in fy_sheets:
113     try:
114         df = pd.read_excel("/tmp/livetable.xlsx", sheet_name=sheet,
skiprows=2, header=0)
115         df.columns = [str(c).strip() for c in df.columns]
116         all_comp = [c for c in df.columns if "All Completions" in c or c ==
"All Completions"]
117         all_start = [c for c in df.columns if "All Starts" in c or c == "All
Starts"]
118         if all_comp and all_start:
119             year_str = sheet.replace("FY_", "").split("-")[0]
120             live_frames.append({
121                 "FY_Start_Year": int(year_str),
122                 "Annual_Completions": pd.to_numeric(df[all_comp[0]],
errors="coerce").sum(),
123                 "Annual_Starts": pd.to_numeric(df[all_start[0]],
errors="coerce").sum()
124             })
125     except:
126         continue
127 live_df = pd.DataFrame(live_frames) if live_frames else None
128 if live_df is not None:
129     live_df["Cal_Year"] = live_df["FY_Start_Year"]
130
131 # — 2.5 BUILD TARGET ENCODING MAPS (To reach R^2 > 0.8)
132 print(" Building hyper-local target encodings...")

```

```

133 # We use the existing production parquet file as our geographic ground truth
    lookup anchor
134 try:
135     anchor_file = pq.ParquetFile("final_production_parquet/dataset.parquet")
136     anchor_df = anchor_file.read(columns=["Price", "Town_City",
    "Postcode"]).to_pandas()
137
138     # 1. Town/City Mean Encoding
139     town_mean = anchor_df.groupby("Town_City")["Price"].agg(["mean", "count"])
140     # Filter out statistically insignificant small samples using global mean
    smoothing
141     global_mean = anchor_df["Price"].mean()
142     town_mean["Town_Avg_Price"] = (town_mean["count"] * town_mean["mean"] + 10
    * global_mean) / (town_mean["count"] + 10)
143     town_lookup = town_mean[["Town_Avg_Price"]].reset_index()
144
145     # 2. Postcode Sector Extraction (e.g., "SW1A 1AA" -> "SW1A 1")
146     anchor_df["Postcode_Sector"] =
    anchor_df["Postcode"].astype(str).str.rsplit(" ", n=1).str[0]
147     postcode_mean = anchor_df.groupby("Postcode_Sector")["Price"].agg(["mean",
    "count"])
148     postcode_mean["Area_Avg_Price"] = (postcode_mean["count"] *
    postcode_mean["mean"] + 5 * global_mean) / (postcode_mean["count"] + 5)
149     postcode_lookup = postcode_mean[["Area_Avg_Price"]].reset_index()
150
151     print(f" Mapped {len(town_lookup)} cities and {len(postcode_lookup)}
    postcode sectors.")
152 except Exception as e:
153     print(f" WARNING: Could not build target encodings ({e}). Using empty
    fallbacks.")
154     town_lookup, postcode_lookup = None, None
155
156 # — 3. PROCESS TRANSACTIONS IN CHUNKS
    

---


157
158 print("\n=== Processing transactions ===")
159
160 writer = None
161 total_rows = 0
162
163 def feature_engineering(df):
164     """Applies target transformations, time tracking, and geographic data
    science enhancements."""
165     df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
166     df = df.dropna(subset=["Price"])
167
168     # CRITICAL CHANGE: Eliminate extreme outliers and luxury anomalies hurting
    R^2
169     df = df[(df["Price"] > 10_000) & (df["Price"] < 5_000_000)]
170
171     # CRITICAL CHANGE: Log-transform target column to normalize variance
172     df["Log_Price"] = np.log1p(df["Price"])
173
174     df["Transfer_Date"] = pd.to_datetime(df["Transfer_Date"], errors="coerce")

```

```

175     df = df.dropna(subset=["Transfer_Date"])
176
177     df["Transaction_Month"] = (df["Transfer_Date"].dt.year * 100 +
df["Transfer_Date"].dt.month).astype(int)
178     df["Transfer_Year"] = df["Transfer_Date"].dt.year.astype(int)
179     df["Transfer_Month"] = df["Transfer_Date"].dt.month.astype(int)
180
181     # Time Seasonality Multipliers
182     df["Quarter"] = df["Transfer_Date"].dt.quarter.astype(int)
183     df["Is_Summer_Peak"] = df["Transfer_Month"].isin([6, 7, 8]).astype(int)
184
185     # Clean strings
186     df["Town_City"] = df["Town_City"].astype(str).str.strip()
187     df["Postcode_Sector"] = df["Postcode"].astype(str).str.rsplit(" ",
n=1).str[0].str.strip()
188
189     # Map Target Encodings
190     if town_lookup is not None:
191         df = df.merge(town_lookup, on="Town_City", how="left")
192     if postcode_lookup is not None:
193         df = df.merge(postcode_lookup, on="Postcode_Sector", how="left")
194
195     # Drop high-cardinality raw text strings and target column (using
Log_Price instead)
196     # Keeping raw features can confuse the AutoML model and bloat memory
197     drop_cols = ["Price", "Transfer_Date", "Transaction_ID", "PAON", "SAON",
"Street",
198                 "Locality", "Record_Status", "Postcode", "Postcode_Sector",
"Town_City"]
199     df = df.drop(columns=drop_cols, errors="ignore")
200
201     # Fill any gaps in target encodings with the global dataset average
202     if "Town_Avg_Price" in df.columns:
203         df["Town_Avg_Price"] = df["Town_Avg_Price"].fillna(500000.0)
204     if "Area_Avg_Price" in df.columns:
205         df["Area_Avg_Price"] = df["Area_Avg_Price"].fillna(500000.0)
206
207     return df
208
209 def process_chunk(df, source_label):
210     global writer, total_rows
211     df.columns = TRANSACTION_COLS
212
213     df = feature_engineering(df)
214
215     # Joins
216     df = df.merge(bank_monthly, on="Transaction_Month", how="left")
217     df = df.merge(gdp_monthly, on="Transaction_Month", how="left")
218     if cpi_lookup is not None:
219         df = df.merge(cpi_lookup, on="Transaction_Month", how="left")
220     if live_df is not None:
221         df = df.merge(live_df[["Cal_Year", "Annual_Completions",
"Annual_Starts"]],

```

```

222         left_on="Transfer_Year", right_on="Cal_Year",
how="left")
223         df = df.drop(columns=["Cal_Year"], errors="ignore")
224
225         table = pa.Table.from_pandas(df, preserve_index=False)
226
227         if writer is None:
228             writer = pq.ParquetWriter(f"{OUT_DIR}/dataset.parquet", table.schema)
229             writer.write_table(table)
230             total_rows += len(df)
231             print(f"    [{source_label}] chunk written - {len(df):,} rows (total so
far: {total_rows:,})")
232
233     # Source 1: process existing parquet
234     print(" Source 1: existing parquet")
235     parquet_file = pq.ParquetFile("final_production_parquet/dataset.parquet")
236     for i, batch in enumerate(parquet_file.iter_batches(batch_size=CHUNK_SIZE)):
237         chunk = batch.to_pandas()
238         # Ensure it maps down to the raw names before feature engineering
239         process_chunk(chunk, f"parquet batch {i+1}")
240
241     # Source 2: process 202304.csv
242     print(" Source 2: 202304.csv")
243     for i, chunk in enumerate(pd.read_csv(
244         "/tmp/transactions_extra.csv",
245         header=None, chunksize=CHUNK_SIZE,
246         on_bad_lines="skip", low_memory=False)):
247         process_chunk(chunk.copy(), f"csv chunk {i+1}")
248
249     if writer:
250         writer.close()
251
252     # — 4. WRITE MLTable

```

```

253
254     print("\n=== Writing MLTable ===")
255     mltable = """$schema:
https://azuremlschemas.azureedge.net/latest/MLTable.schema.json
type: mltable
paths:
  - pattern: ./*.parquet
transformations:
  - read_parquet: {}
"""
261
262     with open(f"{OUT_DIR}/MLTable", "w") as f:
263         f.write(mltable)
264
265     print(f"\n✓ Done! Total rows: {total_rows:,}")
266     print(f"✓ Output: {OUT_DIR}/dataset.parquet")

```